

Winter 390 Presentation 3: Team 5

Fauzan Aryaputra, Paris Bozzuti, Ronaldo Tineo, Emma Smith, Aidan Balagtas

February 2026



Testing On All Benign Cases

Aidan

Making New Splits/Running Model

- Used make_splits.py to make splits 2-5
- Ensured benign cases that were split are not divided between train/test so that there is no data leakage
- Ran base model w/ presentation 3 mandatory changes on new splits
 - No attention analysis updates

Split #: [benign case numbers]

Split 1: [21, 28, 34, 77, 79, 86, 96]

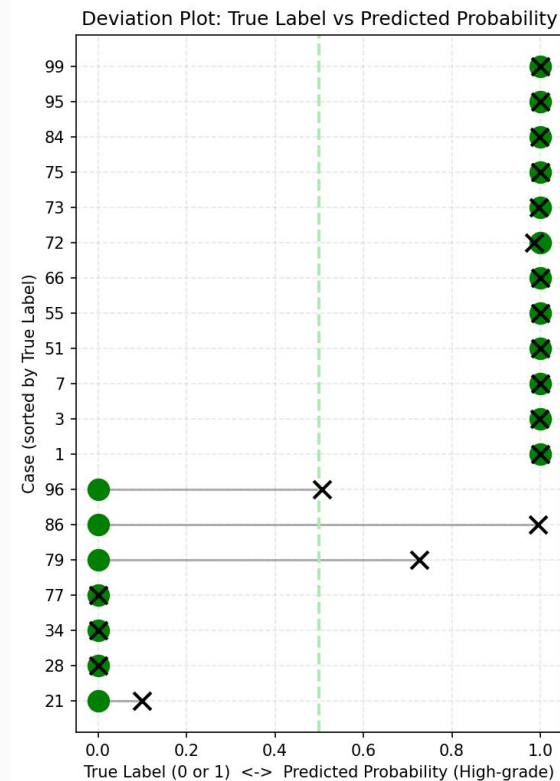
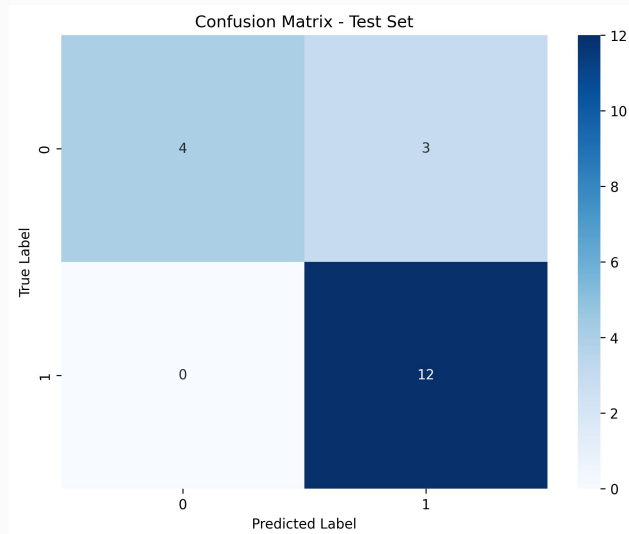
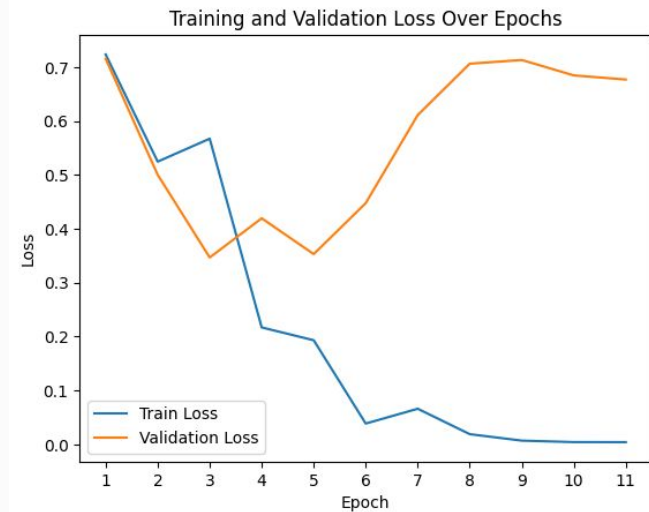
Split 2: [4, 27, 30, 56, 78, 82, 93]

Split 3: [2, 23, 43, 45, 53, 85, 90]

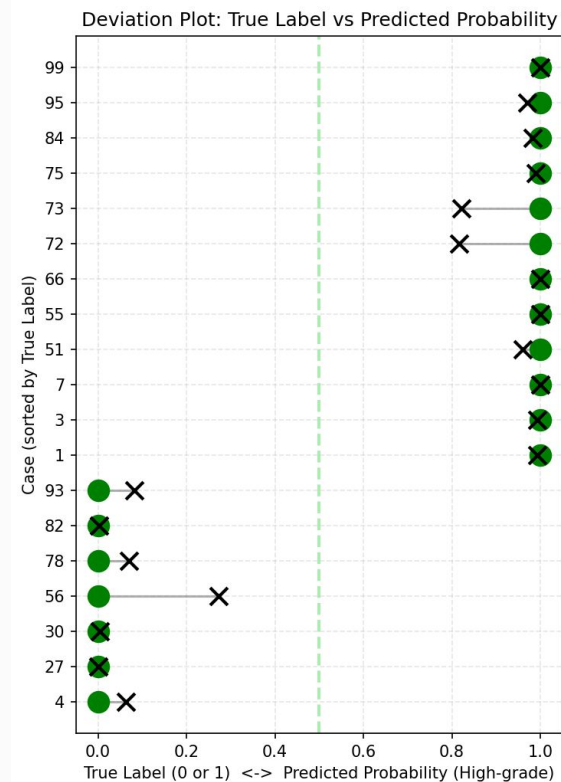
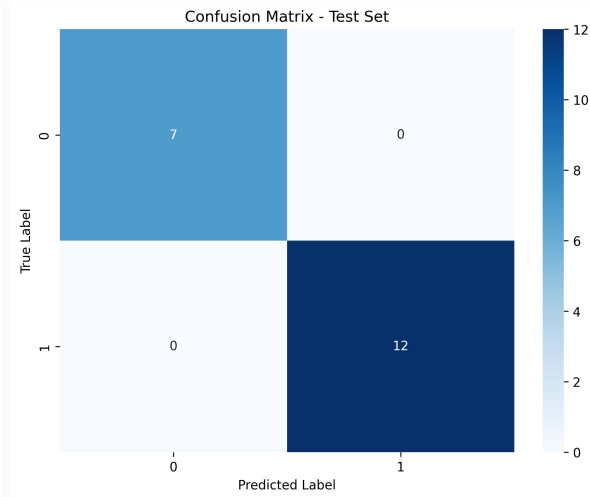
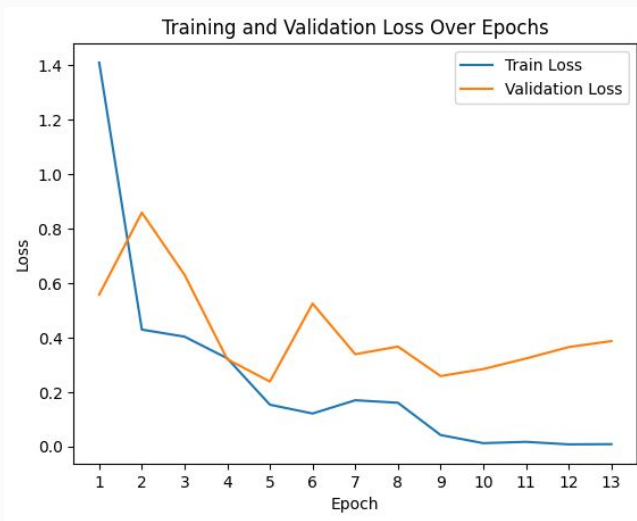
Split 4: [22, 24, 43, 46, 107, 108, 118]

Split 5: [25, 26, 43, 109, 110, 111, 119]

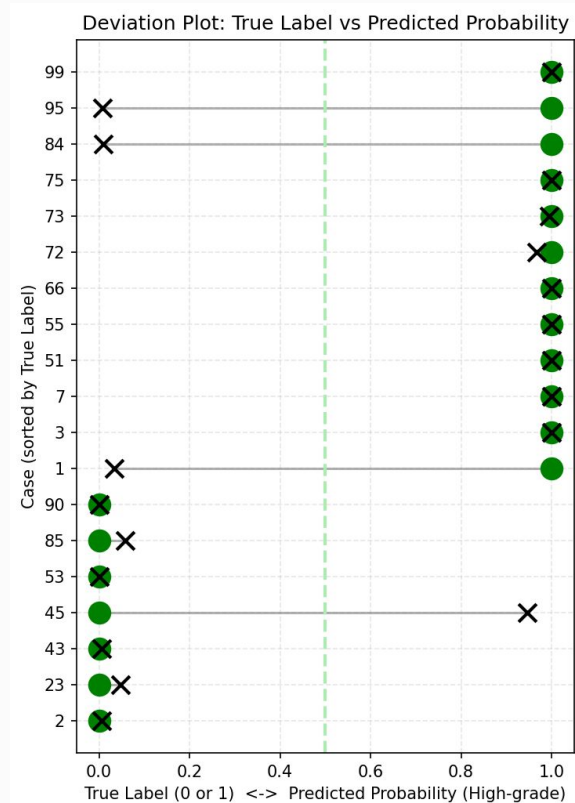
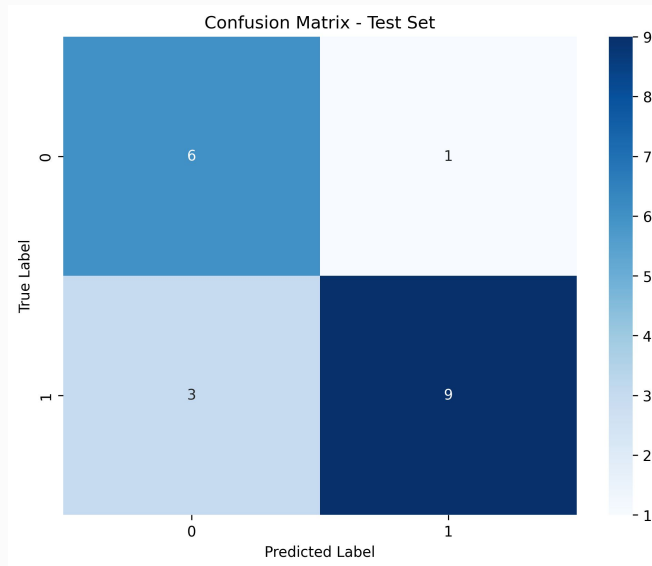
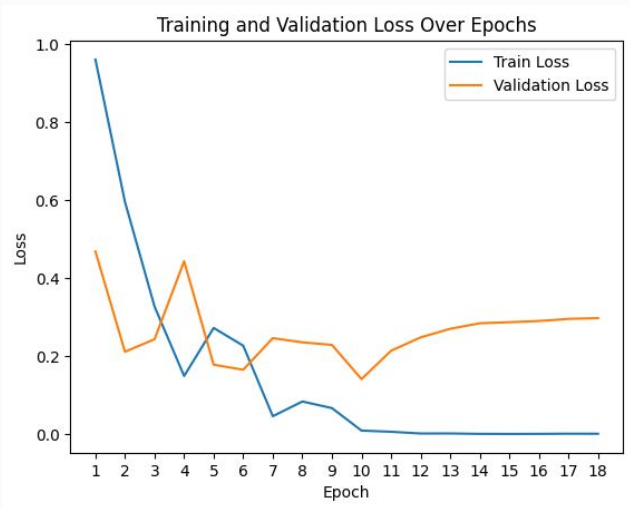
Split 1: Benign Cases 21, 28, 34, 77, 79, 86, 96



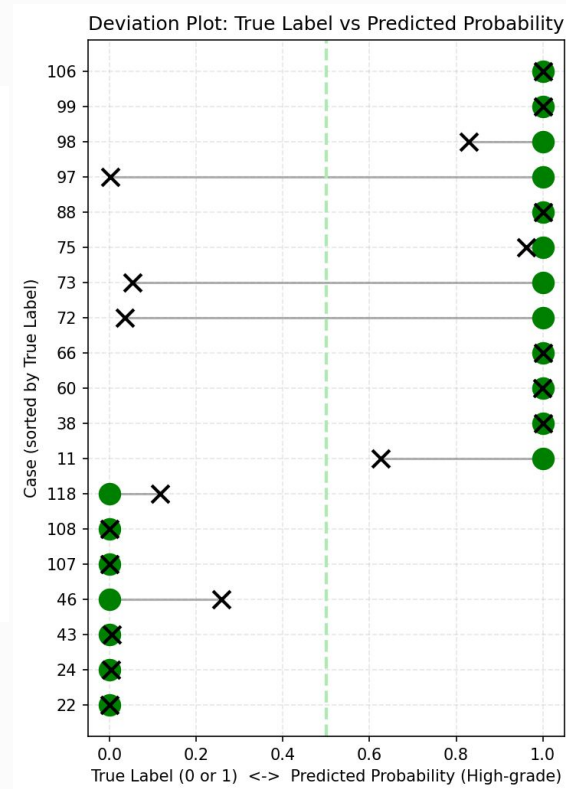
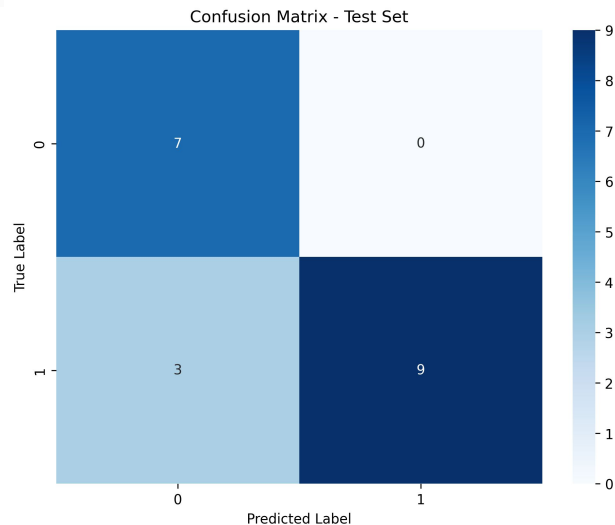
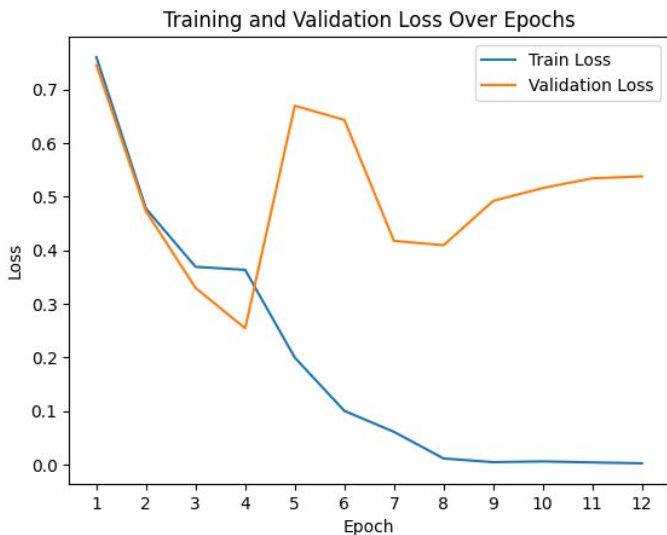
Split 2: Benign Cases 4, 27, 30, 56, 78, 82, 93



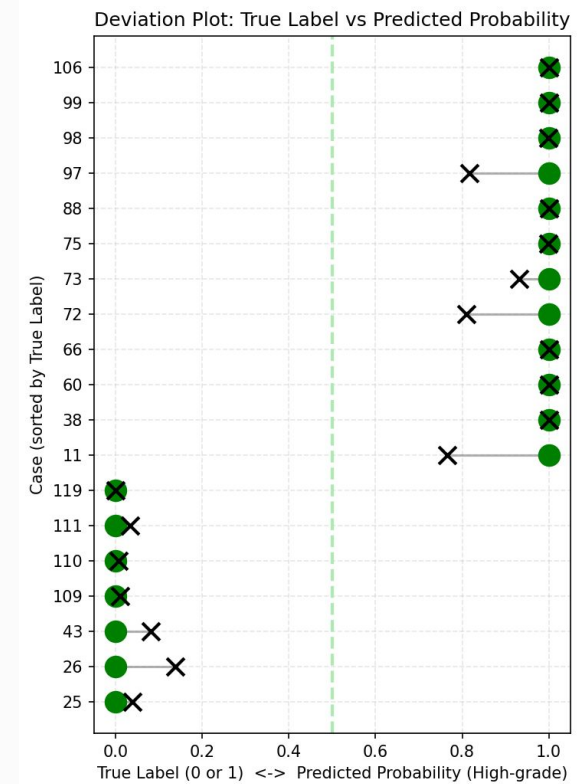
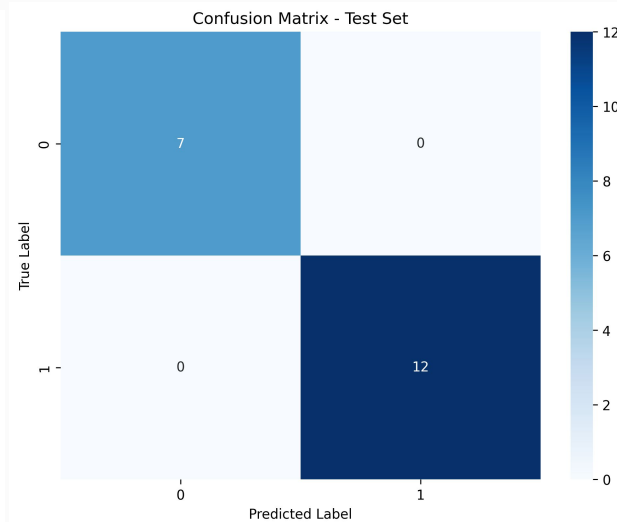
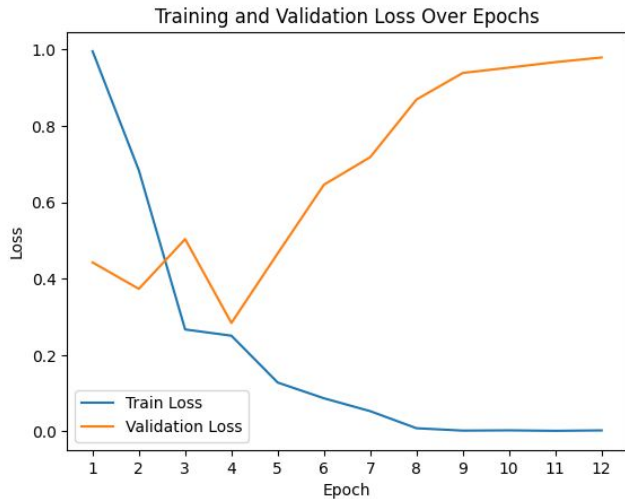
Split 3: Benign Cases 2, 23, 43, 45, 53, 85, 90



Split 4: Benign Cases 22, 24, 43, 46, 107, 108, 118



Split 5: Benign Cases 25, 26, 43, 109, 110, 111, 119



Result Summary

- Misclassified benign cases:
 - 45 (split 3)
 - 79, 86, 96 (split 1)
- Model overfitting after 3-4 epochs in splits 1, 4, 5
- Worse performance on high-grade cases than benign in splits 3, 4

Optuna: Hyperparameter Tuning

Paris

optuna_tuner_paris.py

```
# Create datasets (uses precomputed pooled features)
train_ds = StainBagCasePooledFeatureDataset(
    train_case_dict, train_label_map,
    embeddings_dir=args.embeddings_dir,
    per_slice_cap=args.per_slice_cap,
    max_slices_per_stain=args.max_slices_per_stain,
    shuffle_patches=True,
)

val_ds = StainBagCasePooledFeatureDataset(
    val_case_dict, val_label_map,
    embeddings_dir=args.embeddings_dir,
    per_slice_cap=args.per_slice_cap,
    max_slices_per_stain=args.max_slices_per_stain,
    shuffle_patches=False,
)

# Create data loaders
train_loader = DataLoader(
    train_ds, batch_size=1, shuffle=True,
    num_workers=args.num_workers, pin_memory=True,
    collate_fn=case_collate_fn
)

val_loader = DataLoader(
    val_ds, batch_size=1, shuffle=False,
    num_workers=args.num_workers, pin_memory=True,
    collate_fn=case_collate_fn
)
```

```
def objective(trial, train_data, val_data, args):
    """
    Optuna objective function
    Suggests hyperparameters and returns validation loss
    """
    # Suggest hyperparameters
    trial_params = {
        # (a) Initial learning rate fed into ReduceLROnPlateau scheduler
        'learning_rate': trial.suggest_float('learning_rate', 1e-5, 1e-3, log=True),
        # (b) Weight decay
        'weight_decay': trial.suggest_float('weight_decay', 1e-6, 1e-3, log=True),
        # (c) Dropout in the patch projector
        'coeff_dropout': trial.suggest_float('coeff_dropout', 0.0, 0.5),
        # (d) Dropout in the classifier head
        'classifier_dropout': trial.suggest_float('classifier_dropout', 0.0, 0.5),
        # (e) Class weight for benign class (malignant fixed at 1.0)
        'class_weight_benign': trial.suggest_float('class_weight_benign', 1.0, 5.0),
        # (f) Entropy regularization coefficient for attention diversity
        'entropy_coeff': trial.suggest_float('entropy_coeff', 0.0, 0.5),

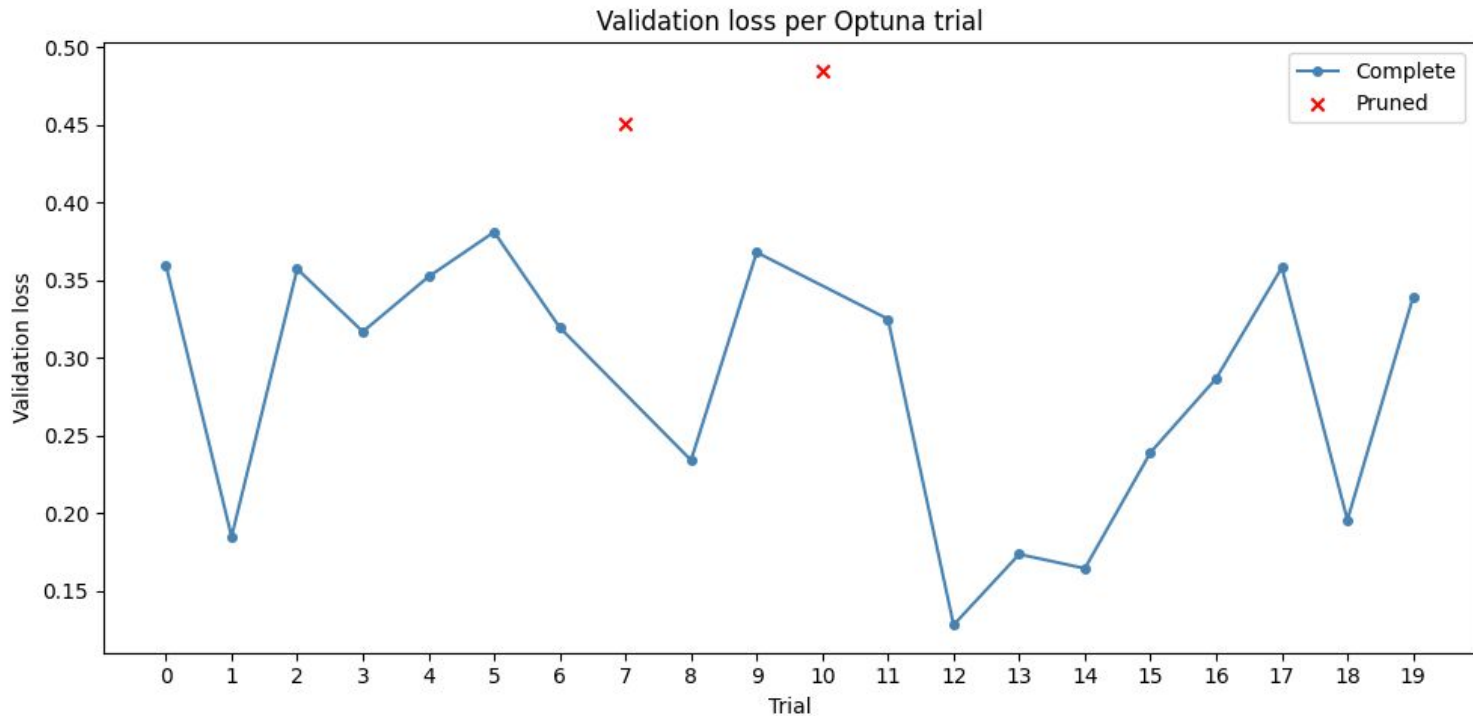
        # Fixed scheduler values
        'use_scheduler': True,
        'scheduler_factor': 0.5,
        'scheduler_patience': 3,
        'scheduler_min_lr': 1e-7,
    }
}
```

optuna_tuner_paris.py

Trials: 20

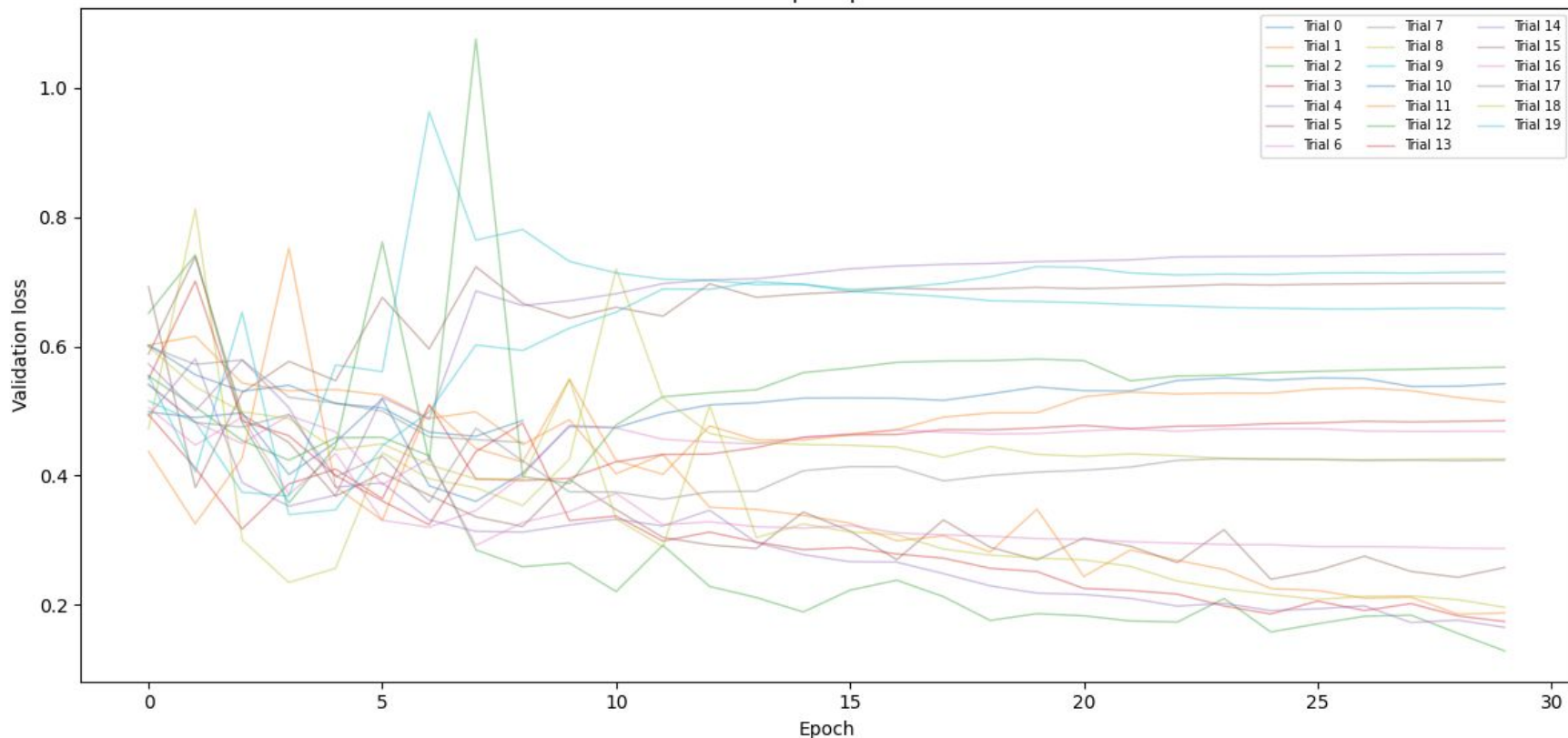
Epochs: 30

Runtime: 20 hours, 33 minutes using GPU



optuna_tuner_paris.py

Validation loss per epoch — all trials



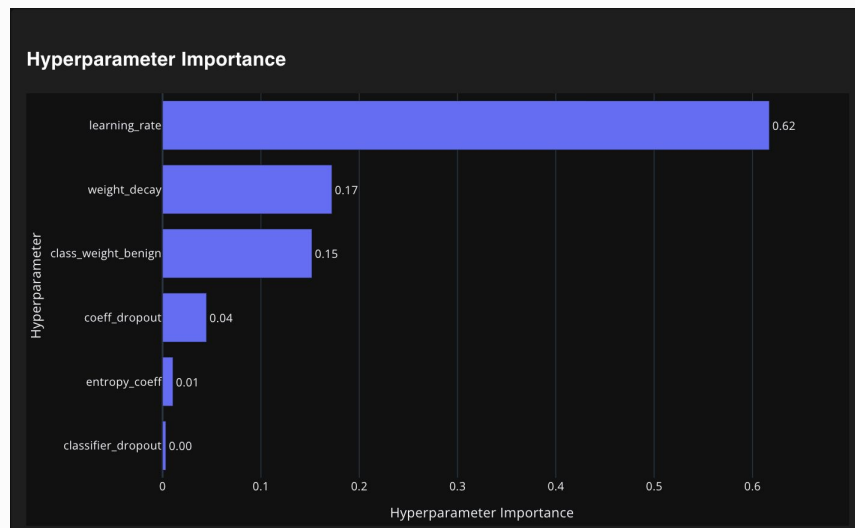
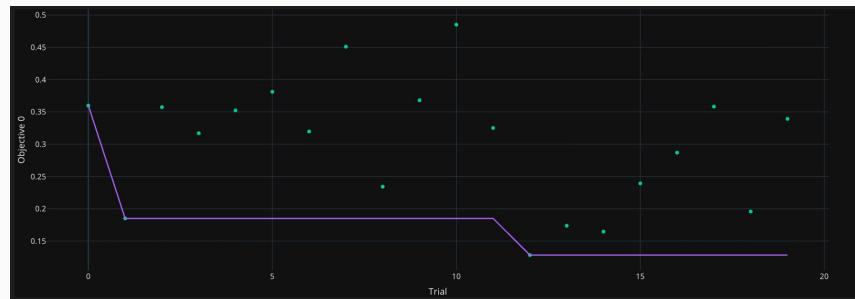
Dashboard

Trial 12 (trial_id=13)

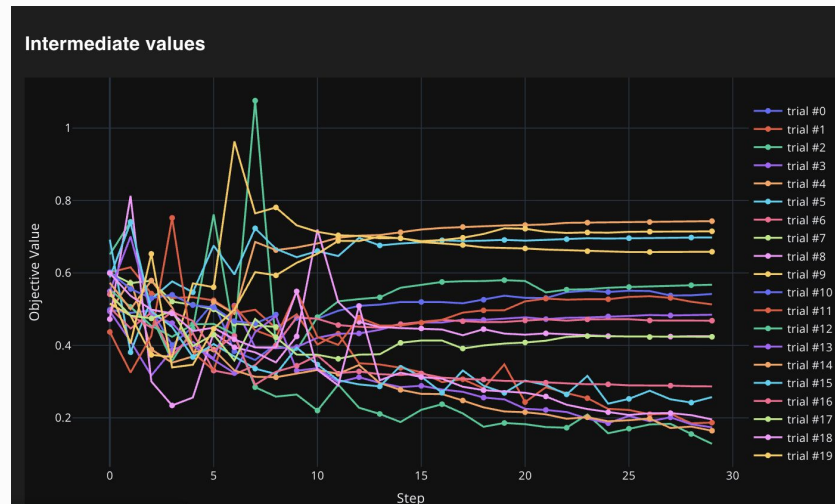
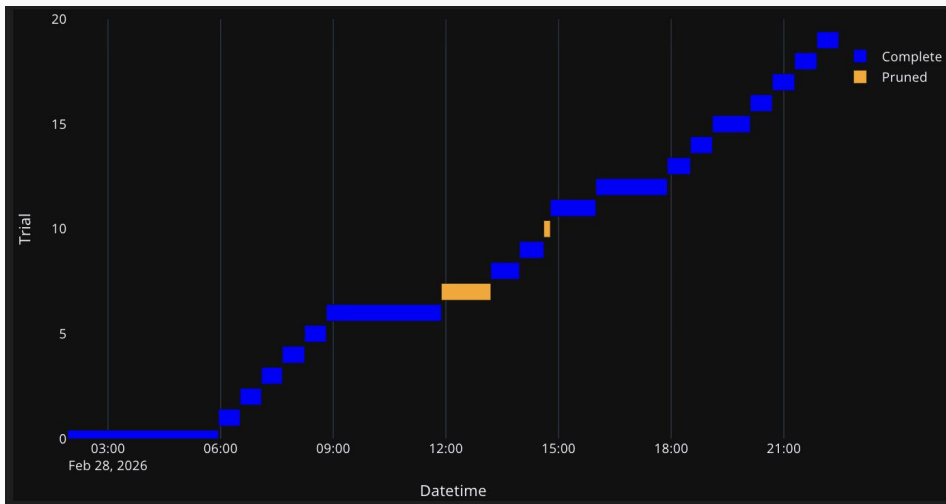
Complete Best Trial

Note

Value	0.12820468937827492
Intermediate Values	24 0.15749522200991017 25 0.17015709782358232 26 0.18179048190359026 27 0.18353057182770605 28 0.15539321791199173 29 0.12820468937827492
Parameter	learning_rate 2.7528862695812924e-05 weight_decay 4.044574059404959e-05 coeff_dropout 0.493388667319079 classifier_dropout 0.3930373854314628 class_weight_benign 4.160322358622171 entropy_coeff 0.4990497566649651
Started At	Sat Feb 28 2026 15:59:39 GMT-0600 (Central Standard Time)
Completed At	Sat Feb 28 2026 17:53:45 GMT-0600 (Central Standard Time)
Duration	6846688
Time Unit	ms
User Attributes	best_epoch 30 best_val_accuracy 0.9411764705882353



Dashboard



Run `~/local/bin/optuna-dashboard sqlite:///optuna.db in /projects/e32998/STAT390_Winter2026/Team5/Code/Code7_OptunaChanges_27Feb_Paris/optuna_20260228_015544`

Optuna Results

```
TRAINING_CONFIG = {  
    'learning_rate': 2.7528862695812924e-05,  
    'weight_decay': 4.044574059404959e-05,  
    'coeff_dropout': 0.493388667319079,          # patch projector dropout  
    'dropout': 0.3930373854314628,              # classifier head dropout  
    'class_weights': [4.160322358622171, 1.0],  
    'entropy_coeff': 0.4990497566649651,  
}
```

/projects/e32998/STAT390_Winter2026/Team5/Code/Code7_OptunaChanges_27Feb_Paris/optuna_20260228_015544/best_config.py

```
{  
    "best_value": 0.12820468937827492,  
    "best_trial_number": 12,  
    "best_epoch": 30,  
    "best_val_accuracy": 0.9411764705882353,  
    "n_trials": 20,  
    "datetime_completed": "2026-02-28 22:27:48"  
}
```

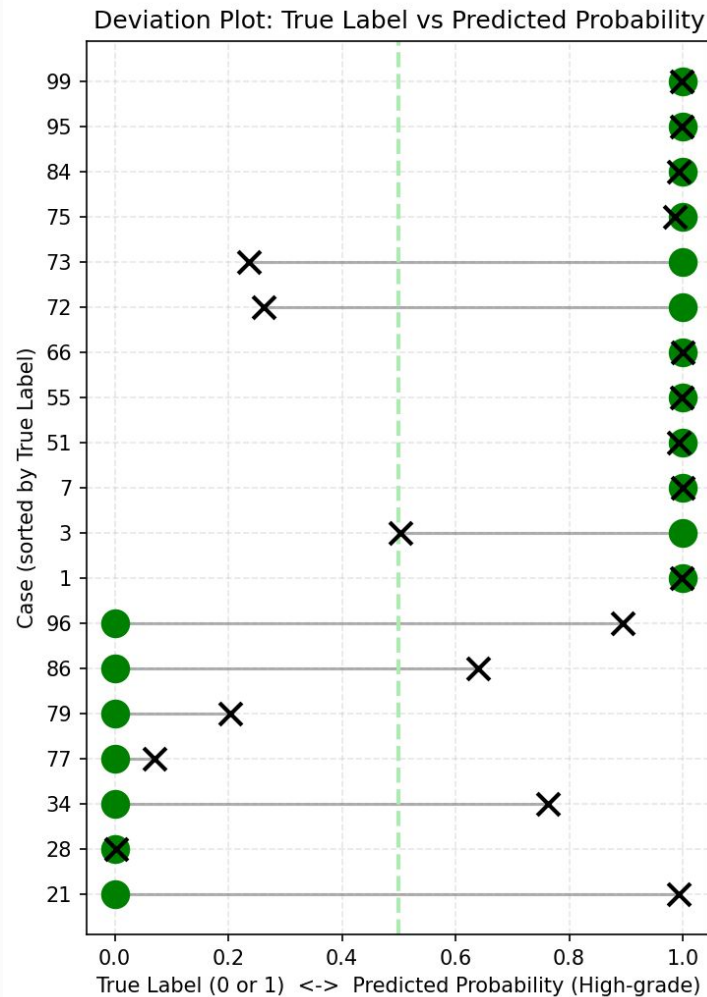
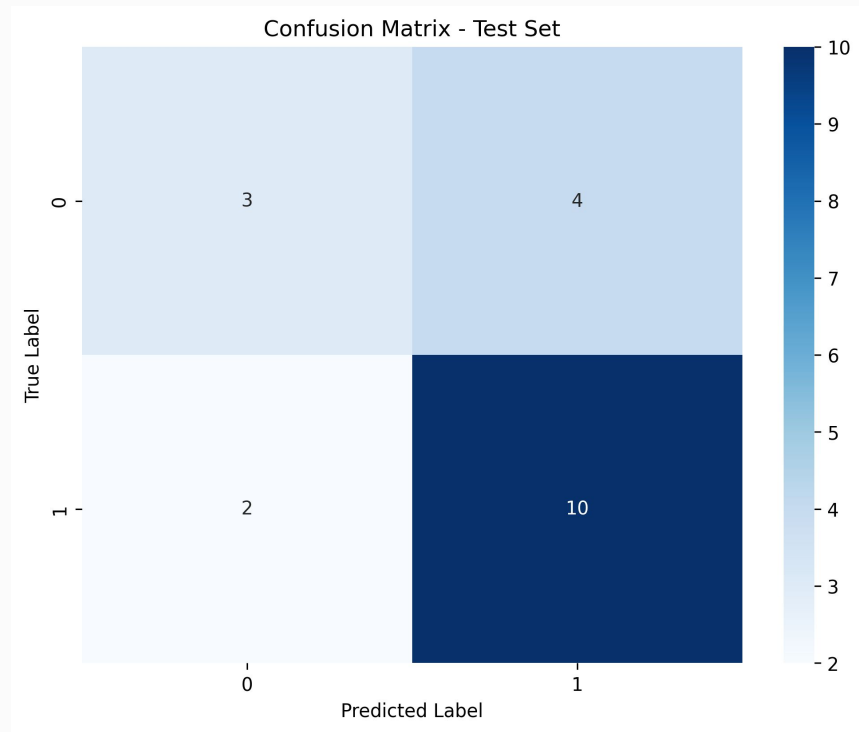

Model Results

Test Results:

Test Loss: 0.7095

Test Accuracy: 0.6842

/projects/e32998/MIL_training/final_runs/run_20260301_125350/results.txt



Theory: Considering Slice-Level Removal

Fauzan

Current Model

The current model uses a level of slice attention within the hierarchical model

1) Patch attention:

$$a_{s,l,p} = f_{patch}(z_{s,l,p})$$

$$\alpha_{s,l,p} = \frac{e^{a_{s,l,p}}}{\sum_p e^{a_{s,l,p}}}$$

$$h_{s,l} = \sum \alpha_{s,l,p} z_{s,l,p}$$

2) Slice attention:

$$b_{s,l} = f_{slice}(h_{s,l})$$

$$\beta_{s,l} = \frac{e^{b_{s,l}}}{\sum_l e^{b_{s,l}}}$$

$$g_s = \sum \beta_{s,l} h_{s,l}$$

Slice Level Relationship + Argument Response

- 1) “Peaky patch attention” is a within-slice property, not a class-evidence guarantee
 - a) “Peaky” means that one patch has a much larger attention score than the others:

$$a_{s,l,p^*} - a_{s,l,p} \gg 0$$

- b) This can occur because:
 - i) There is an outlier patch that the attention MLP likes
 - ii) The slice has low diversity so one patch becomes dominant
 - iii) Attention is learning what is salient, but not necessarily what is predictive
 - c) Further, this means that a peaky patch attention **does not logically imply** “strong class evidence”

Slice Level Relationship + Argument Response

- 2) Slice attention depends on the slice embedding, but the slice embedding does not encode peakiness
- a) If the patch attentions are peaky, then the peaked patch dominates the embedding, but there is no indication of how “confident” the model is about the class of the case
 - b) Slice attention then uses:

$$b_{s,l} = f_{slice}(h_{s,l})$$

$$\beta_{s,l} = \frac{e^{b_{s,l}}}{\sum_l e^{b_{s,l}}}$$

- c) Unless f_{slice} is explicitly designed to be a monotone function of peakiness (which it isn't, it's a MLP), the model has no reason to map a peaky patch attention to a high slice score

Slice Level Relationship + Argument Response

- 3) I offer some counterexamples to refute the statement:
 - a) All slices peaky
 - i) Slice attention compares these $h_{s,l}$ vectors
 - ii) Even if every slice is peaky, slice attention can be uniform, peaky, or anything depending on f_{slice}
 - b) Slice is peaky on the *wrong* patch
 - i) Let one slice contain an artifact patch that gets a huge attention score
 - ii) If f_{slice} has learned artifacts are non-predictive, it will assign a low logit despite peaky patch attention
 - c) True evidence requires slice-level competition
 - i) Let two slices have peaky patch attention, but only one is discriminative
 - ii) Slice attention is exactly the mechanism to choose between them

Considering Removal

Should we remove the slice-level hierarchy?

- 1) Slice attention solves a different problem than patch attention
- 2) Provides protection against “artifact winners” at patch level
 - a) Acts as a filter against noise at the patch level
- 3) Provides adaptive pooling across heterogeneous slides
 - a) Some cases have the melanoma concentrated in few slices, others are diffuse
 - b) Slice attention learns a data-dependent mixture
- 4) Mean pooling hard-codes one behavior for all cases, increasing bias in peaky cases and variance in diffuse cases
- 5) Attending to all patches at once runs into the signal-to-noise ratio issue discussed in class

Theory: Resolution as a Hyperparameter

Fauzan

Computational Complexity

- 1) CNN forward complexity dominates and scales about quadratically with resolution

- a) A standard convolutional layer costs:

$$FLOPs_l = \Theta(h_l w_l \cdot k_l^2 \cdot c_{l-1} \cdot c_l)$$

- b) Since $h \cdot w$ scales like H^2 , total CNN cost per patch is approximately:

$$FLOPs \text{ per patch} = \Theta(H^2)$$

- 2) Attention / pooling is usually cheaper and scales linearly in number of patches

- a) At each pooling level, compute an attention MLP per instance and then a weighted sum

- b) Resolution has no impact on computational complexity here

Small Pathological Regions

- 1) After resizing to H, a pathological region of physical diameter r becomes:

$$r' = r \cdot \frac{H}{H_{orig}}$$

- 2) If r' drops below a few pixels, the signal is effectively destroyed (aliased/blurred)
- 3) Lower resolution (smaller H) increases the chance that tiny cancer signals become indistinguishable from background
- 4) Higher resolution (larger H) preserves small structures, but at H^2 compute cost
- 5) Needle in haystack effect:
- a) If the cancer occupies a fraction p of the patch area, downsampling tends to shrink contrast roughly proportional to p because the discriminative pixels get averaged with background

Bias / Variance Tradeoff

1) Bias

- a) Downsampling removes high-frequency detail, which increases approximation error:

$$\textit{Bias}(H) \uparrow \text{ as } H \downarrow$$

2) Variance

- a) Higher resolution increases effective input dimension and sensitivity to nuisance detail:

$$\textit{Var}(H) \uparrow \text{ as } H \uparrow$$

3) So test error takes the form:

$$\textit{TestError}(H) = \textit{Bias}(H)^2 + \textit{Var}(H) + \textit{Noise}$$

Is Resolution a Hyperparameter?

Yes. It controls:

- 1) Compute budget
- 2) Information retained
- 3) Bias variance tradeoff

However, we can get the benefits of balancing the bias-variance tradeoff that resolution allows us to control through via attention downstream.

This allows us to run the frozen feature extractor once with high resolution, and save those low-level embeddings to be used repeatedly later.

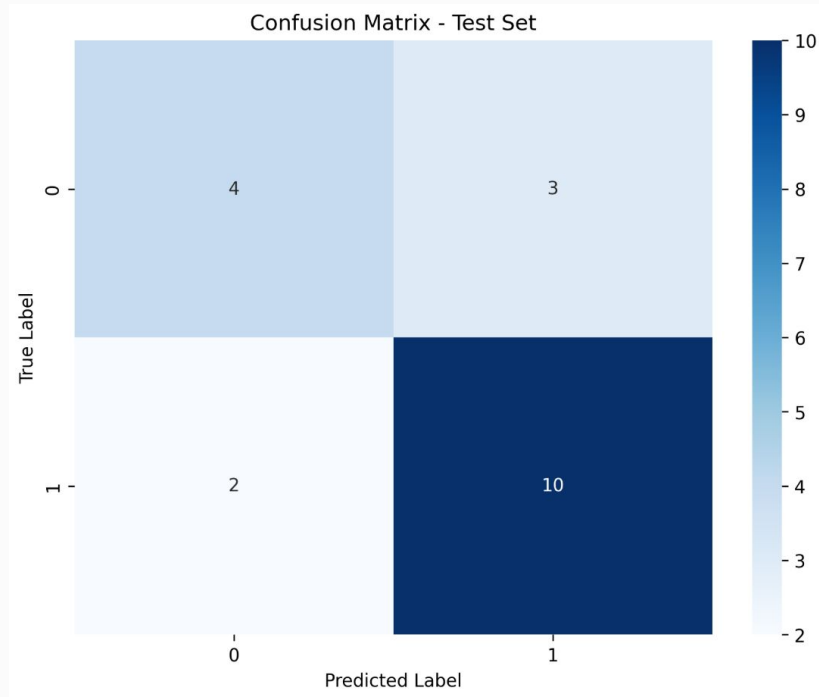
Just need to ensure that resolution is high enough, as we cannot “learn back” information lost by low resolution.

Patch-Level Predictions

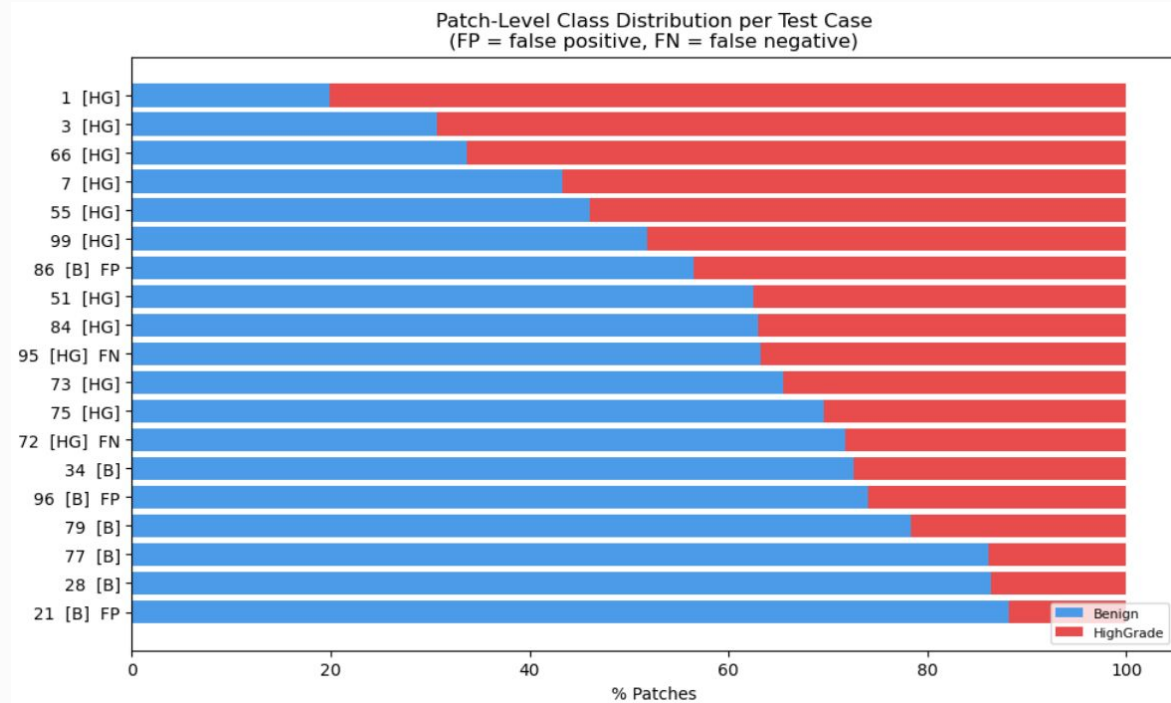
Emma, Ronaldo

Mandatory Changes: Patch Level Analysis

- For every test case, take raw embeddings and pass them individually through patch_projector, which compresses each one down to 512 dimensions
- Then apply the classifier weights directly to each compressed embedding
 - producing a Benign/HighGrade probability for every patch independently

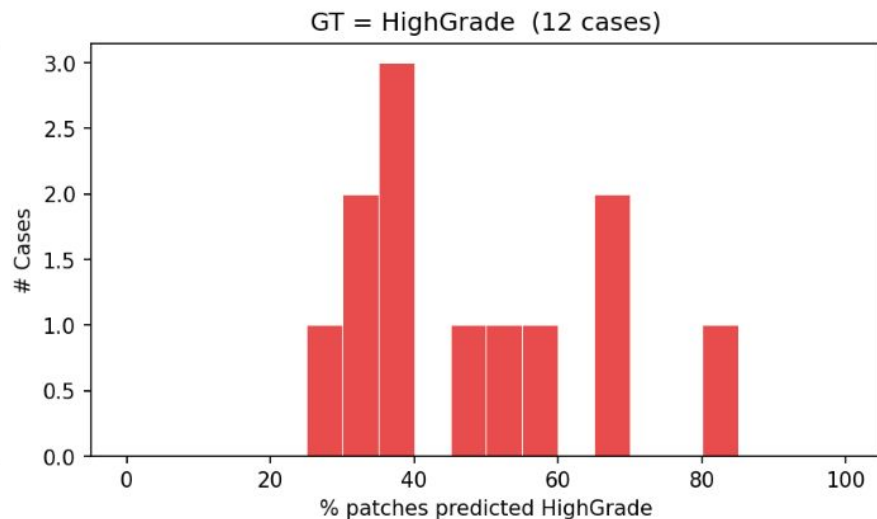
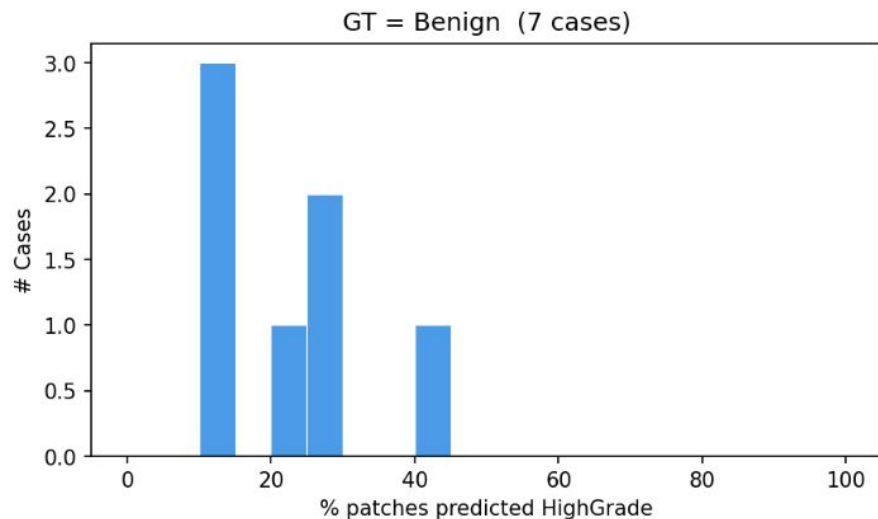


Mandatory Changes: Patch Level Analysis

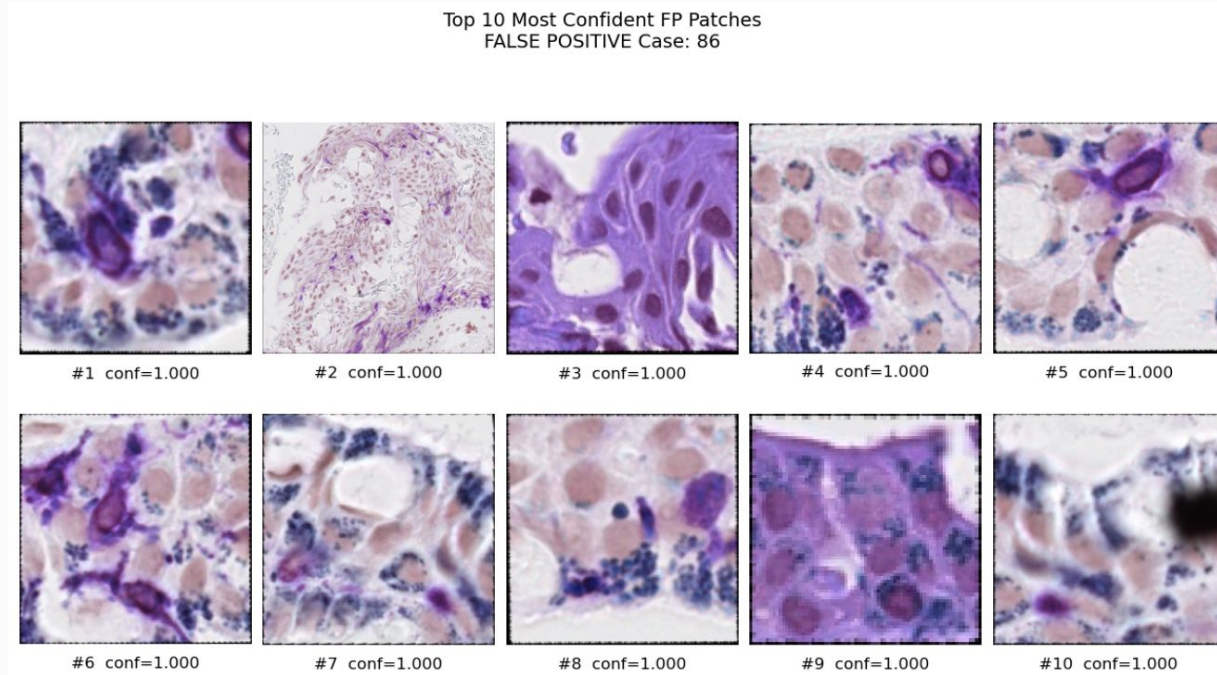


Mandatory Changes: Patch Level Analysis

Distribution of HighGrade-predicted patch fraction by GT class

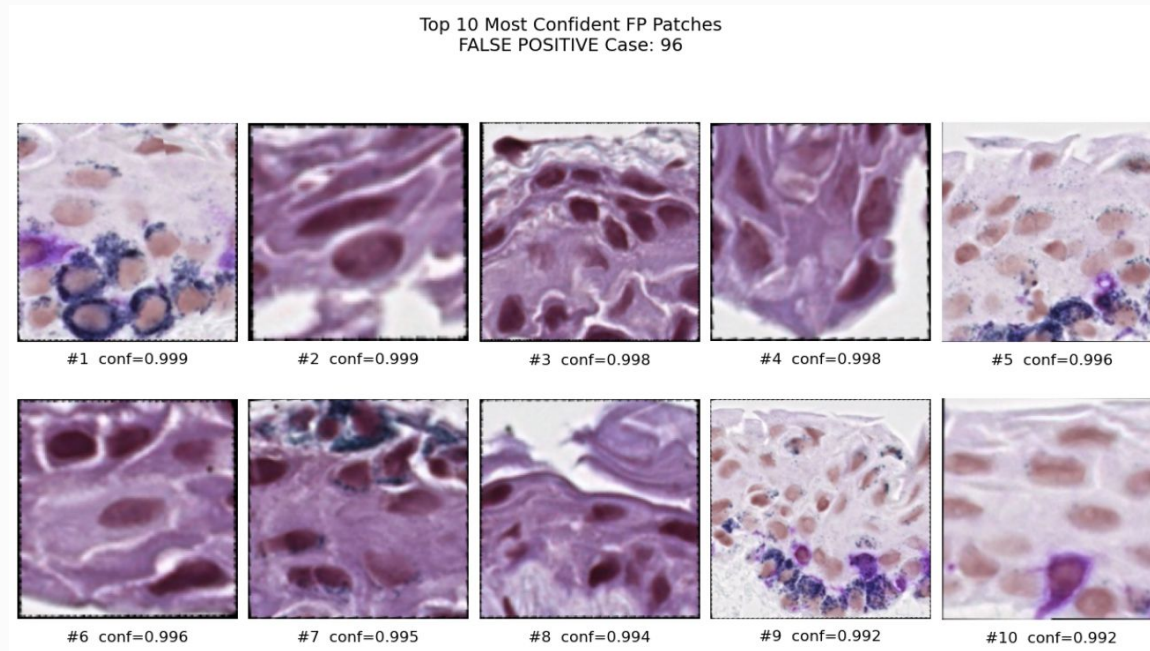


Mandatory Changes: Patch Level Analysis



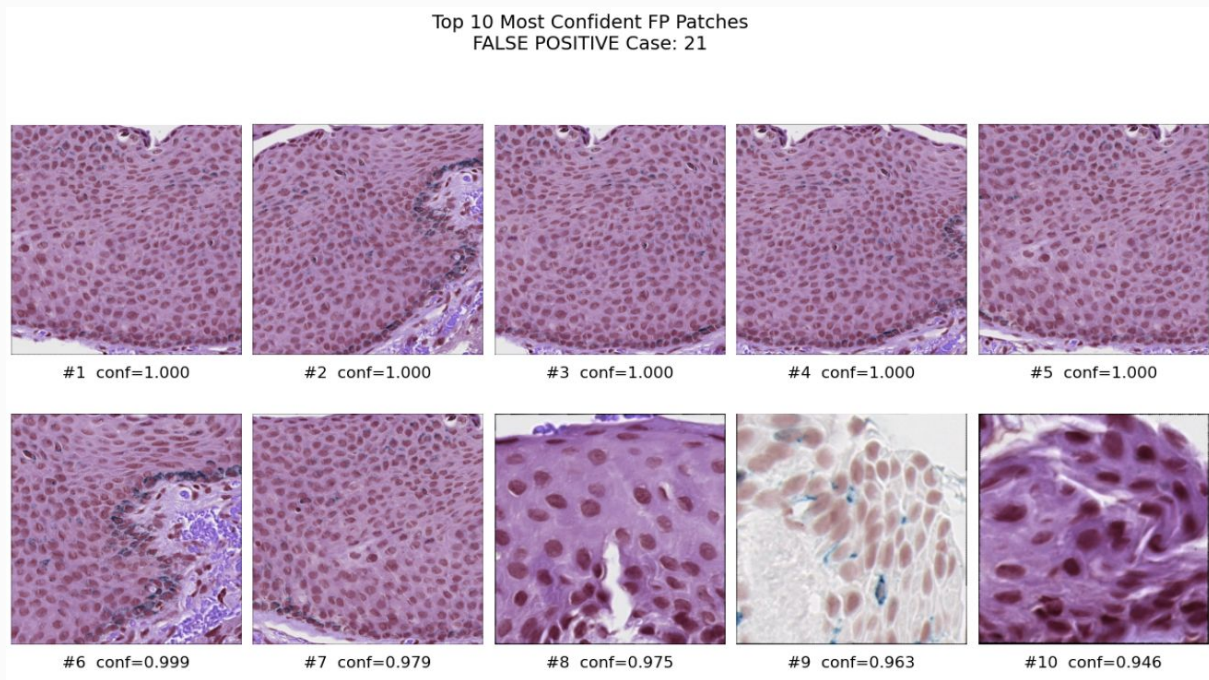
- Some patches with known issue of goblet cells (5, 7)
- Irregular “streaky” pattern (2, 3)

Mandatory Changes: Patch Level Analysis



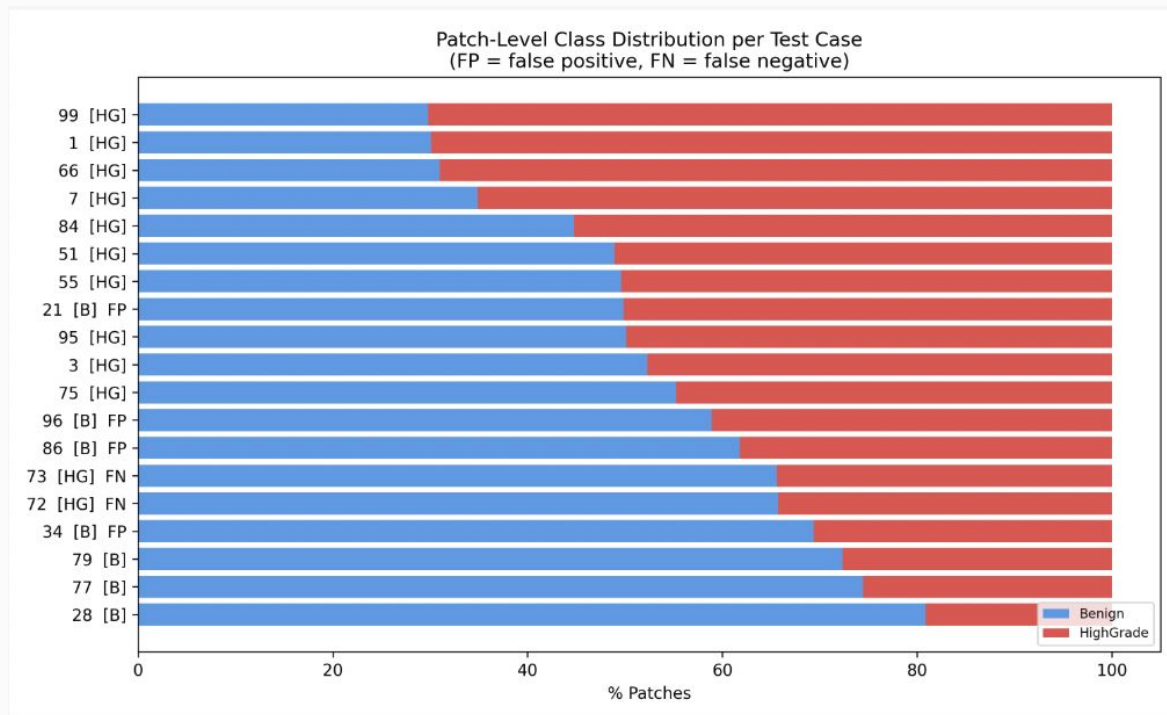
- Seems like many of these patches aren't spanning the entire epithelium (2, 3, 4, 6, 7, 8)

Mandatory Changes: Patch Level Analysis



- Learning the thick epithelial h&e as HighGrade (1, 2, 3, 4, 5, 6, 7)
- See a bit of streaky irregular cell patterns (9, 10)

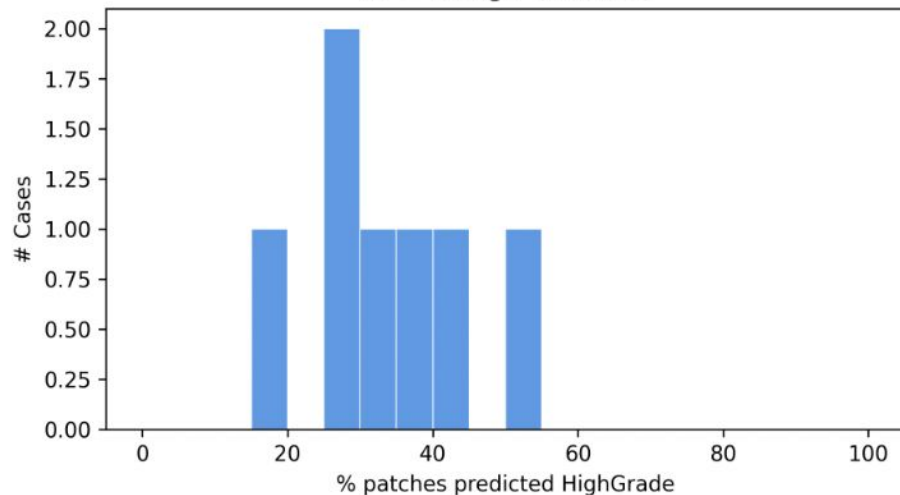
Optuna Changes: Patch Level Analysis



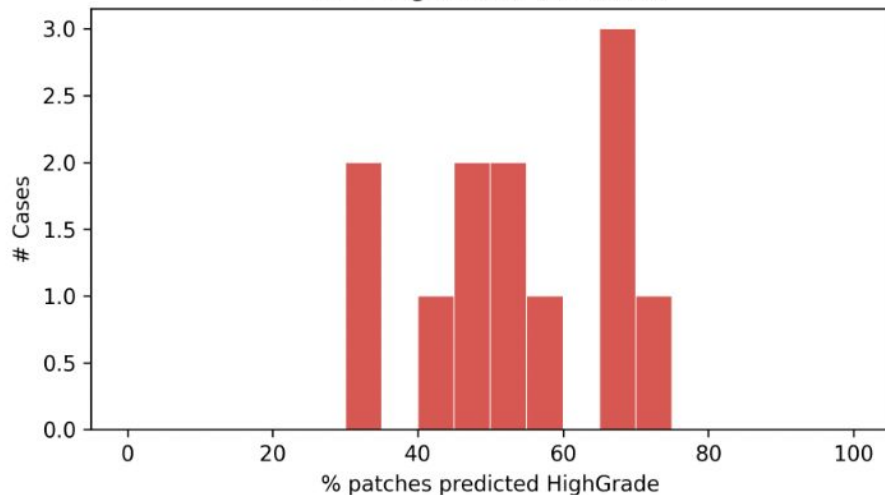
Optuna Changes: Patch Level Analysis

Distribution of HighGrade-predicted patch fraction by GT class

GT = Benign (7 cases)



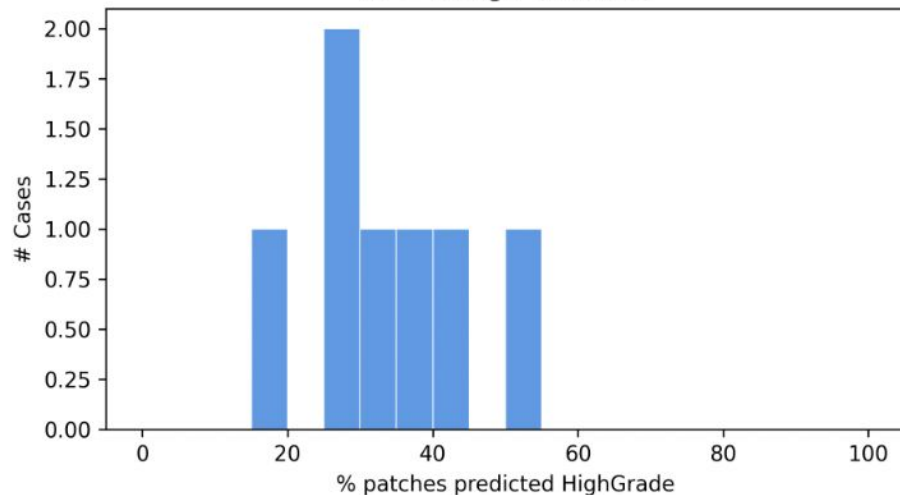
GT = HighGrade (12 cases)



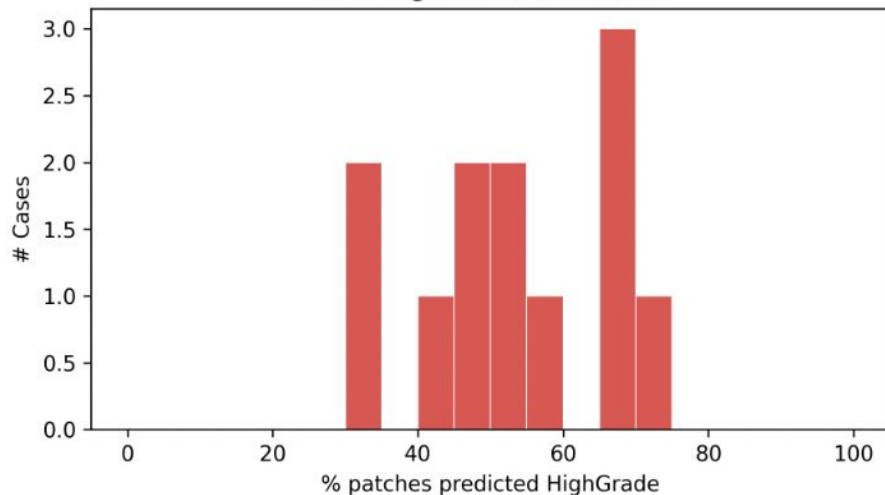
Optuna Changes: Patch Level Analysis

Distribution of HighGrade-predicted patch fraction by GT class

GT = Benign (7 cases)

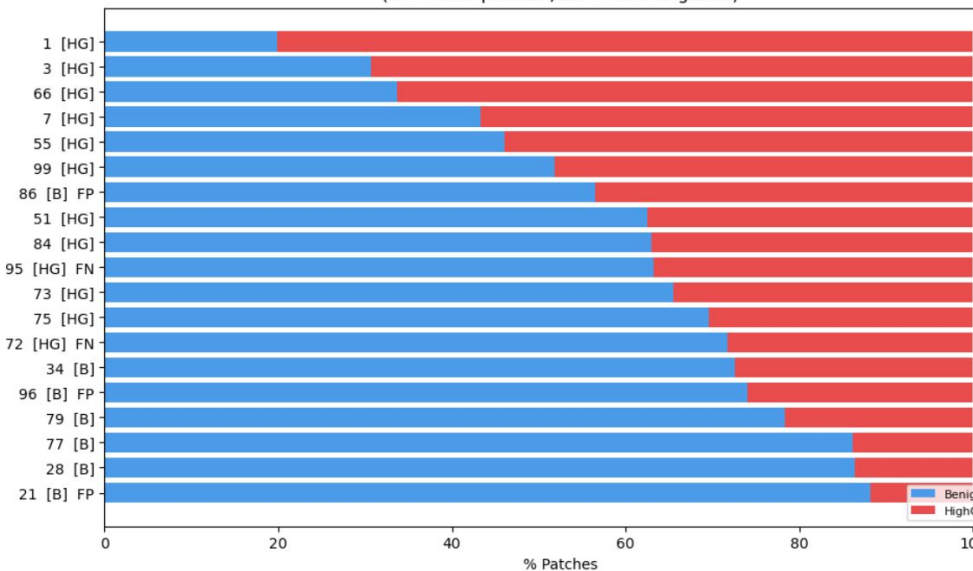


GT = HighGrade (12 cases)



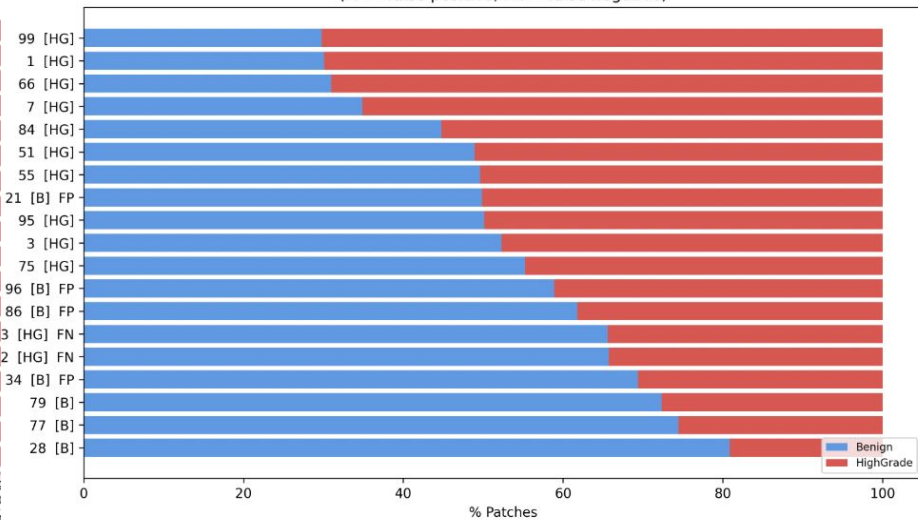
Compare and Contrast: Patch Level Analysis

Patch-Level Class Distribution per Test Case
(FP = false positive, FN = false negative)



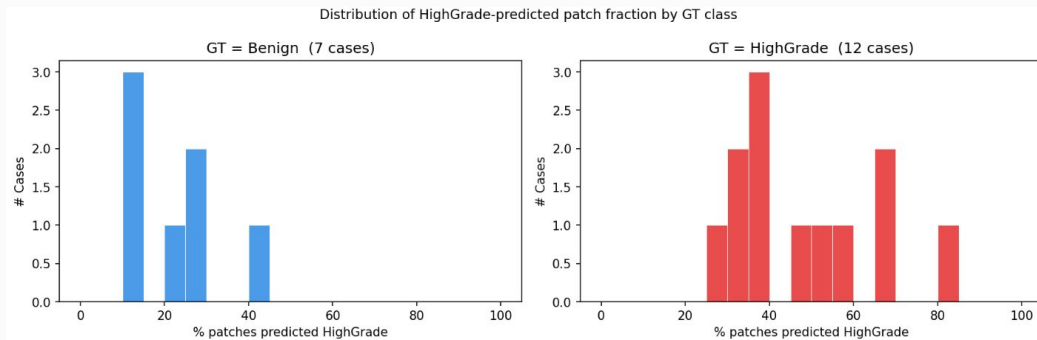
/projects/e32998/STAT390_Winter2026/Team5/Code/Code8_PatchLevelAnalysis_2Mar_Emma

Patch-Level Class Distribution per Test Case
(FP = false positive, FN = false negative)

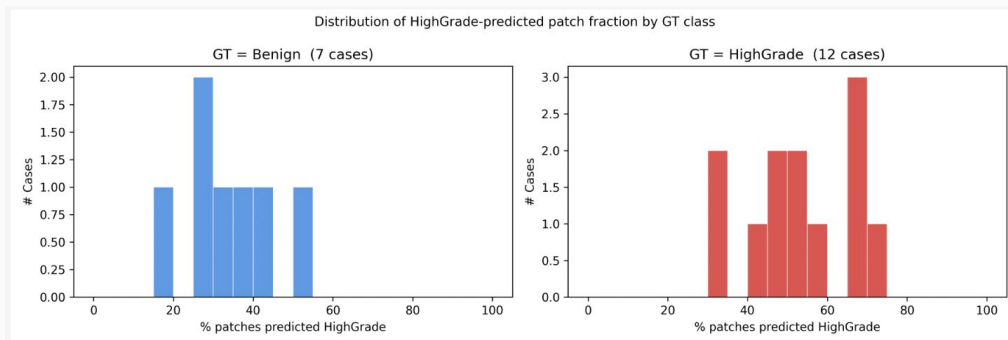


/projects/e32998/STAT390_Winter2026/Team5/Code/Code9_PatchLevelAnalysis_Optuna_Ronaldo
projects/e32998/MIL_training/final_runs/run_20260304_101546

Compare and Contrast: Patch Level Analysis

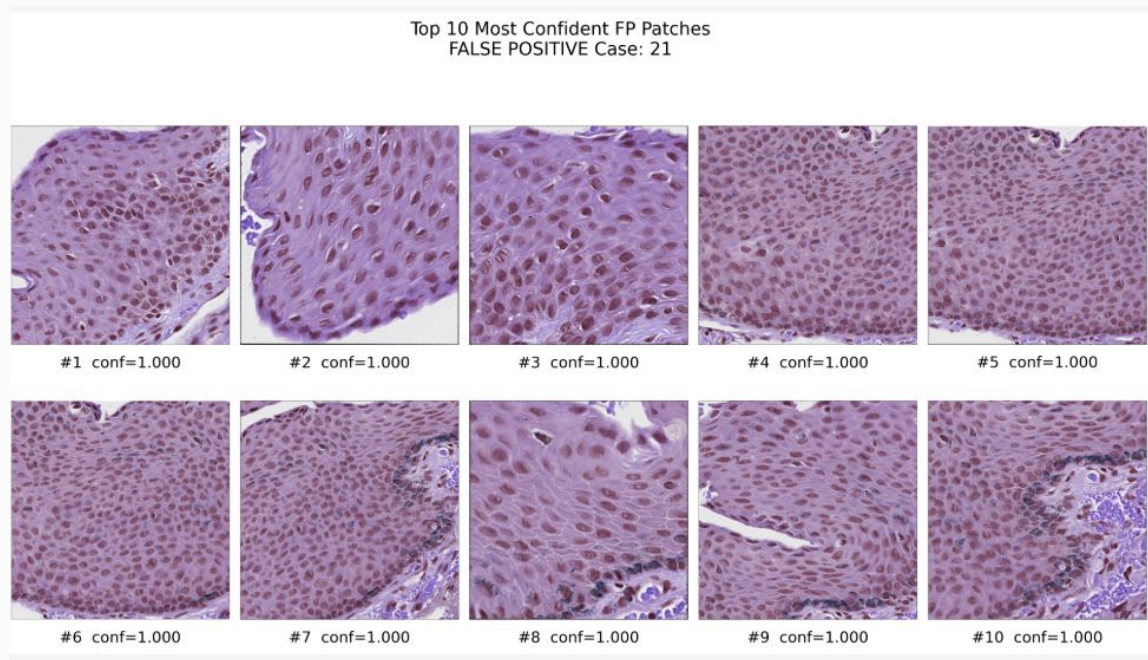


/projects/e32998/STAT390_Winter2026/Team5/Code/Code8_PatchLevelAnalysis_2Mar_Emma



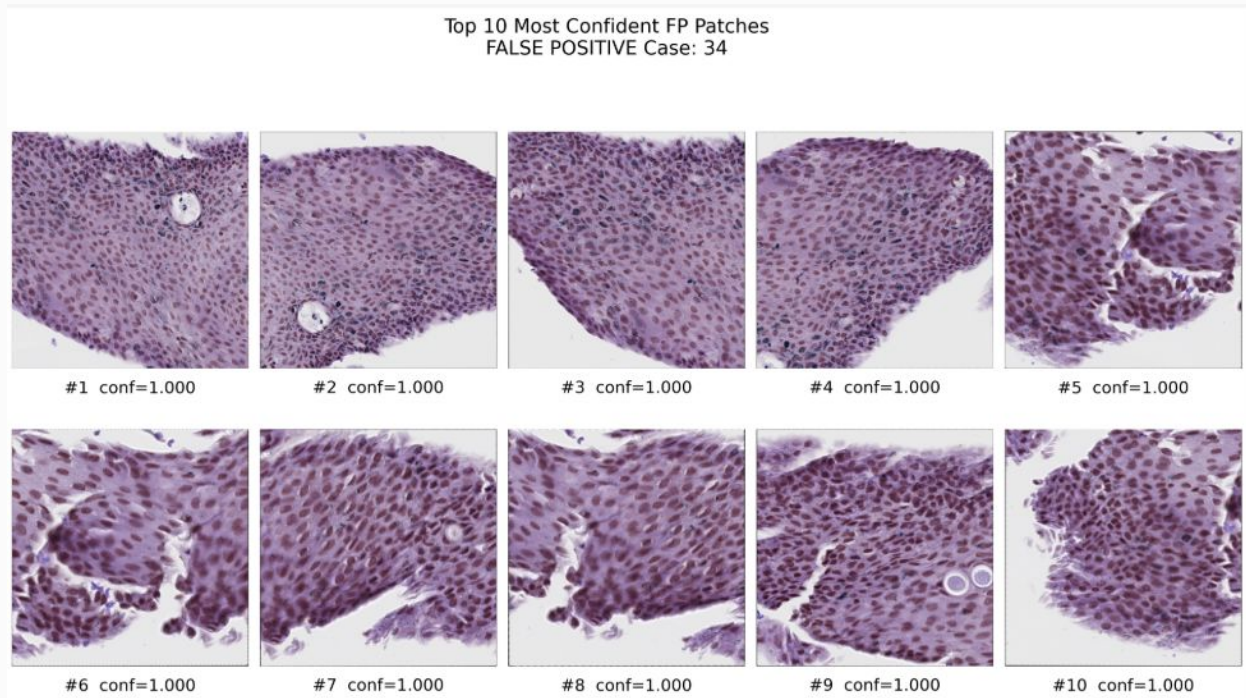
/projects/e32998/STAT390_Winter2026/Team5/Code/Code9_PatchLevelAnalysis_Optuna_Ronaldo
projects/e32998/MIL_training/final_runs/run_20260304_101546

Mandatory Changes: Patch Level Analysis



/projects/e32998/STAT390_Winter2026/Team5/Code/Code9_PatchLevelAnalysis_Optuna_Ronaldo
projects/e32998/MIL_training/final_runs/run_20260304_101546

Mandatory Changes: Patch Level Analysis



/projects/e32998/STAT390_Winter2026/Team5/Code/Code9_PatchLevelAnalysis_Optuna_Ronaldo
projects/e32998/MIL_training/final_runs/run_20260304_101546